Comprehensive Tutorial: Anomaly Detection using ResNet50 Feature Extraction

1. Introduction

This tutorial explains how to implement an unsupervised anomaly detection system in images leveraging deep feature extraction from a pre-trained ResNet50 model. The system compares extracted features from test images against a memory bank of normal image features to detect anomalies at both image and pixel levels. This approach is useful in quality inspection, surveillance, and medical imaging applications.

---

2. System Overview

The system performs the following key functions:

1. **Feature Extraction:** Uses ResNet50's intermediate layers to extract deep feature maps representing normal images.
2. **Memory Bank Creation:** Builds a database of feature vectors from normal images, serving as the reference for anomaly detection.
3. **Anomaly Scoring:** Compares test image features to the memory bank using nearest neighbor distance metrics to score anomaly likelihood.
4. **Pixel-Level Localization:** Generates pixel-level anomaly maps for precise defect localization.
5. **Visualization:** Displays anomaly scores and heatmaps overlayed on test images.

---

3. Prerequisites

3.1 Software & Libraries

- **Python 3.7+**
- **PyTorch 1.8+** (for deep learning and feature extraction)
- **Torchvision** (for pre-trained ResNet50)
- **NumPy** (for numerical operations)
- **Matplotlib** (for plotting results)
- **scikit-learn** (for nearest neighbor search)
- **tqdm** (for progress visualization)
- **PIL (Pillow)** (for image loading)
- **OpenCV** (for visualization and image processing)

3.2 Hardware Requirements

- GPU recommended for faster feature extraction (NVIDIA CUDA-enabled)
- CPU fallback possible but slower

---

4. Step-by-Step Explanation

4.1 Environment Setup and Model Initialization

First, install dependencies and import necessary libraries:

pip install torch torchvision numpy matplotlib scikit-learn tqdm pillow opencv-python

```
import torch
import torchvision.models as models
import torchvision.transforms as transforms
import numpy as np
from PIL import Image
from sklearn.neighbors import NearestNeighbors
import matplotlib.pyplot as plt
import cv2
from tqdm import tqdm
```

Explanation:

- torchvision.models provides pre-trained ResNet50.
- sklearn.neighbors.NearestNeighbors will help with efficient nearest neighbor search.
- tqdm displays progress bars for loops.

4.2 Feature Extraction Function

Extract deep features from the ResNet50 intermediate layer (e.g., layer before the final classifier):

```
class ResNet50FeatureExtractor(torch.nn.Module):
    def __init__(self):
        super().__init__()
        resnet = models.resnet50(pretrained=True)
        # Use all layers except the final fully connected layer
        self.feature_extractor = torch.nn.Sequential(*list(resnet.c
        self.feature_extractor.eval()  # Set to evaluation mode

    def forward(self, x):
        with torch.no_grad():
            features = self.feature_extractor(x)
        return features

# Initialize extractor
extractor = ResNet50FeatureExtractor()
```

Explanation:

- We remove the final classification layer, extracting rich spatial feature maps useful for anomaly detection.
- eval() disables dropout and batch norm updates for consistent feature extraction.

4.3 Preprocessing Images

Transform input images to the required format:

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],  # ImageNet st
                         std=[0.229, 0.224, 0.225]),
])

def load_image(image_path):
    image = Image.open(image_path).convert('RGB')
    return transform(image).unsqueeze(0)  # Add batch dimension
```

Explanation:

- Resizes input images to 224x224 (ResNet50 input size).
- Normalizes pixel values using ImageNet means and std devs.
- Adds batch dimension for model input.

4.4 Building the Memory Bank of Normal Features

Extract and store features from a dataset of normal images:

```
def build_memory_bank(normal_image_paths):
    memory_bank = []

    for img_path in tqdm(normal_image_paths, desc='Building memory
        img_tensor = load_image(img_path)
        features = extractor(img_tensor).squeeze(0)  # Remove batch
        # Flatten spatial features to vectors
        features = features.permute(1, 2, 0).reshape(-1, features.s
        memory_bank.append(features)

    # Concatenate all feature vectors
    memory_bank = np.concatenate(memory_bank, axis=0)
    return memory_bank
```

Explanation:

- Features are extracted for each image and reshaped to vectors for nearest neighbor comparison.
- All features are concatenated to form the memory bank.

4.5 Nearest Neighbor Search for Anomaly Detection

Compare test image features to memory bank to compute anomaly scores:

```
def anomaly_score(test_image_path, memory_bank, k=5):
    img_tensor = load_image(test_image_path)
    features = extractor(img_tensor).squeeze(0)
    h, w, c = features.shape[1], features.shape[2], features.shape[
    features = features.permute(1, 2, 0).reshape(-1, c).cpu().numpy

    nbrs = NearestNeighbors(n_neighbors=k).fit(memory_bank)
    distances, _ = nbrs.kneighbors(features)
    scores = distances.mean(axis=1)  # Average distance to k neighb

    # Reshape scores back to spatial map
    score_map = scores.reshape(h, w)
    return score_map
```

Example usage:

score_map = anomaly_score('test_image.jpg', memory_bank)

Explanation:

- Each spatial feature vector in the test image is compared to its nearest neighbors in the memory bank.
- Higher average distance indicates greater anomaly likelihood.
- Output is a spatial anomaly score map.

4.6 Visualization of Anomaly Heatmap

Overlay anomaly heatmap on the original image:

```python
def visualize_anomaly(image_path, score_map):
    original = cv2.imread(image_path)
    original = cv2.resize(original, (score_map.shape[1], score_map.

    heatmap = cv2.applyColorMap(np.uint8(255 * score_map / score_ma
    overlay = cv2.addWeighted(original, 0.6, heatmap, 0.4, 0)

    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1)
    plt.title("Anomaly Heatmap")
    plt.imshow(cv2.cvtColor(heatmap, cv2.COLOR_BGR2RGB))
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.title("Overlay on Image")
    plt.imshow(cv2.cvtColor(overlay, cv2.COLOR_BGR2RGB))
    plt.axis('off')
    plt.show()
```

Explanation:

- Heatmap visually highlights regions with high anomaly scores.
- Overlay combines the heatmap with the original image for intuitive localization.

---

5. Key Concepts Explained

5.1 ResNet50 Feature Extraction

Uses convolutional layers' output before classification for a rich spatial representation of images.

5.2 Memory Bank

A large collection of feature vectors from normal samples, enabling comparison for anomaly detection.

5.3 Nearest Neighbor Distance Metric

Calculates similarity between test features and normal features; high distances imply anomalies.

5.4 Pixel-Level Anomaly Localization

Provides fine-grained maps indicating which parts of an image are anomalous.

---

6. Potential Improvements

1. **Feature Aggregation:** Use multi-layer features or attention mechanisms to improve representation.
2. **Deep Metric Learning:** Train a dedicated model to improve feature discrimination.
3. **Dynamic Memory Bank:** Update memory bank online with new normal data.
4. **Real-Time Detection:** Optimize for faster inference on edge devices.

5. **Integration with Alert Systems:** Automatically flag detected anomalies for downstream processes.

---

7. Conclusion

This anomaly detection system effectively leverages deep features from a pre-trained ResNet50 model and a memory bank of normal features for robust detection and localization of anomalies. It provides a strong baseline for unsupervised anomaly detection in images, suitable for industrial inspection, medical imaging, and security monitoring.

Final Code Repository

https://github.com/thinkrobotics/Anomaly-Detection-via-Contour-Segmentation